



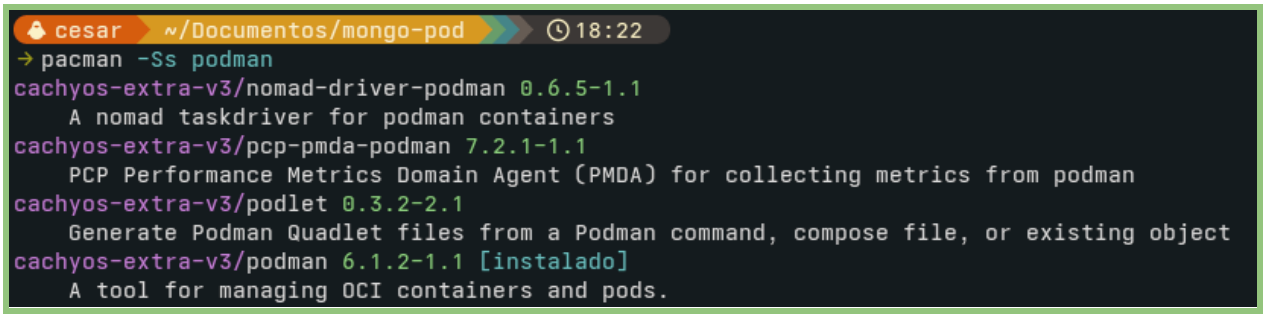
Como crear un contenedor en Podman con almacenamiento persistente en un sistema basado en Arch Linux

Materia: Base de Datos NoSQL

César Ruíz De La Cruz
22020809

23 de septiembre del 2026

Como primer paso deberemos instalar Podman en nuestro sistema; camo *CachyOS* está basado en Arch Linux debemos utilizar la utilidad *pacman* para ver si el paquete está disponible en los repositorios:



Al estar disponible, nos facilita la instalación, solo deberemos ejecutar el comando:



Para crear un contenedor con variables de entorno personalizadas, podemos crear contenedores de manera declarativa; Ayudándonos de la herramienta *podman-compose* y un archivo *compose.yml*.

Para esto, necesitaremos instalar *podman-compose*:



Una vez instalado, deberemos dirigirnos a una carpeta, en donde declararemos el contenedor y almacenamos de manera persistente los datos de este:



Crearemos una carpeta en donde se guardaran los datos:

```
mkdir data
```

Ahora procederemos a crear el archivo *docker-compose.yml* (No importa el nombre, solo la extensión); En el cual pondremos lo siguiente:

```yaml
services:
  mongo:
    image: docker.io/mongodb/mongodb-community-server:latest
    container_name: mongo-pod
    volumes:
      - ./data:/data/db:z
```

En donde:

- Declaramos un servicio llamado: *mongo*
- Con la imagen de: *mongo-community-server:latest*
- Al contenedor lo llamamos: *mongo-pod*
- Y por último le montamos al volumen *./data*(La carpeta que creamos) la ruta que regularmente se guarda en */data/db/* de la imagen de mongo; La *:z* refiere a *SELinux* (Es un módulo de seguridad que aplica políticas de control de acceso para regular cómo los procesos interactúan con los recursos del sistema).

Para levantar el contenedor deberemos ejecutar el comando *podman compose up -d* en la misma carpeta en donde estaba el archivo *.yml*:



Para comprobar que si este activo deberemos ejecutar el comando *podman ps:*

```
cesar ~/Documentos/mongo-pod 19:03
→ podman ps
CONTAINER ID  IMAGE                                          COMMAND     CREATED      STATUS      PORTS       NAMES
699ed598fdd1  docker.io/mongodb/mongodb-community-server:latest  mongod      3 hours ago  Up 3 hours  27017/tcp   mongo-pod
```

*** Para que mongo escriba en la carpeta *data/* (La que creamos) deberemos darle permisos sobre la carpeta; Para fácil:

```
cesar ~/Documentos/mongo-pod 19:03
→ sudo chmod -R 777 data
```

Para ejecutar mongo, deberemos ejecutar el comando:

```
podman exec -it mongo-pod mongosh
```

```
cesar ~/Documentos/mongo-pod 19:08
→ podman exec -it mongo-pod mongosh
Current Mongosh Log ID: 6ab4785fe315b8365f8ce5af
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.6.0
Using MongoDB:          8.3.11
Using Mongosh:          2.6.0
mongosh 2.12.0 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-09-23T22:15:23.736+00:00: Access control is not enabled for the database. Read and write access to data and configura
   2026-09-23T22:15:23.736+00:00: We suggest setting the contents of sysfsFile to 0.
   2026-09-23T22:15:23.736+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
------

test>
```

Ahora crearemos una colección de prueba y agregaremos datos:

```
test> use prueba
switched to db prueba
prueba> db.prueba.insertOne({Saludo: "hello"})
{
  acknowledged: true,
  insertedId: ObjectId('6ab478c8e315b8365f8ce5b0')
}
prueba>
```

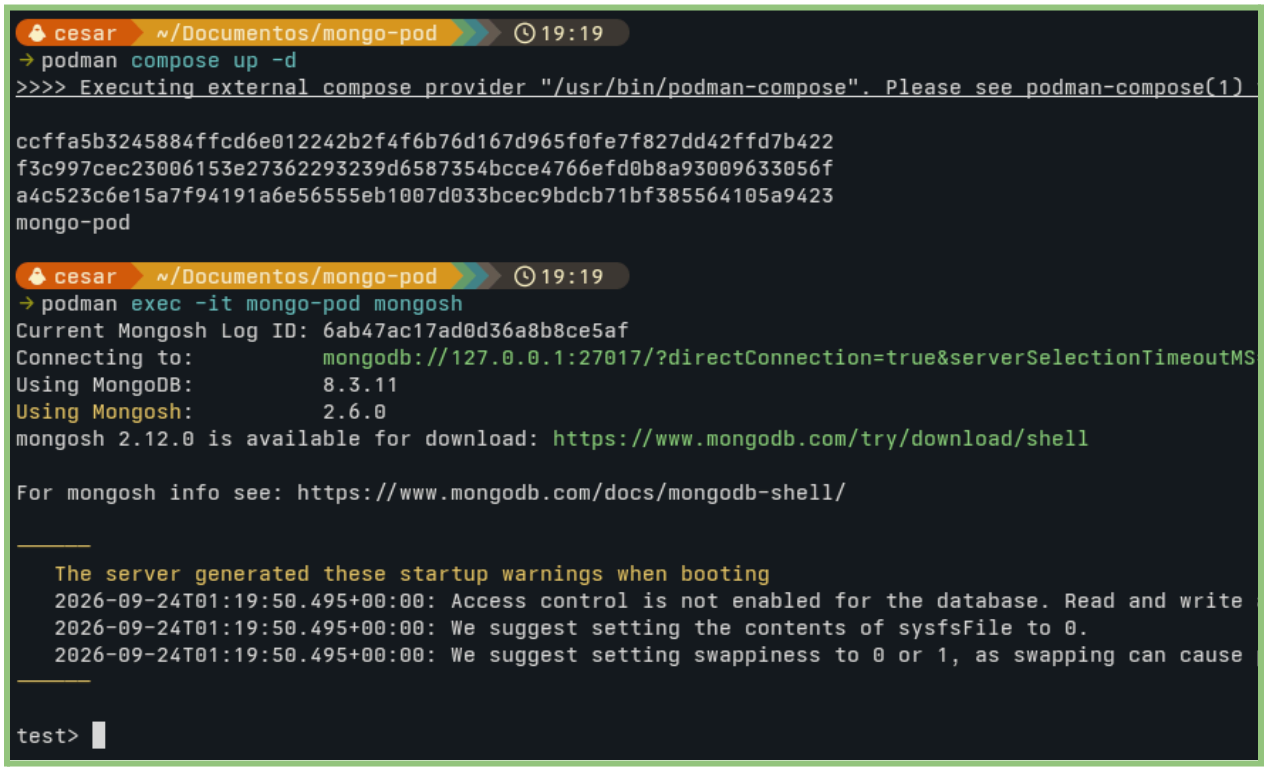
Ahora nos salimos y apagamos el contenedor con el comando:

```
podman compose down
```

y checamos que este apagado:



Nos metemos de nuevo y consultamos los datos de nuestra colección:



```
test> use prueba
switched to db prueba
prueba> show dbs
admin   40.00 KiB
config  60.00 KiB
local   72.00 KiB
prueba  72.00 KiB
prueba>
```

```
prueba> db.prueba.find()
[
  { _id: ObjectId('6ab44fdda1d28ee3948ce5b0'), saludos: 'Prueba' },
  { _id: ObjectId('6ab478c8e315b8365f8ce5b0'), Saludo: 'hello' }
]
prueba>
```

Ahora solo listamos la carpeta data para ver si ahí están los datos de la colección prueba:

```
cesar ~/Documentos/mongo-pod 21:14
→ ls data
drwx------     - 100999 23 sep 16:15  .mongodb
drwx------     - 100999 23 sep 19:19  _tmp
drwx------     - 100999 23 sep 21:14  diagnostic.data
drwx------     - 100999 23 sep 19:20  journal
.rw-------  37k 100999 23 sep 19:19  _mdb_catalog.wt
.rw-------  25k 100999 23 sep 21:12  collection-1baf613d-6f38-4979-bf3d-3c503c566aab.wt
.rw-------  20k 100999 23 sep 19:19  collection-bc7220cd-5524-4d30-aa71-9d87dbcf9f09.wt
.rw-------  37k 100999 23 sep 19:20  collection-c1dac2fb-476d-446c-9155-fdac0131cc26.wt
.rw-------  37k 100999 23 sep 21:11  collection-f1f10f0c-7581-4f2f-a004-83adaa4994b9.wt
.rw-------  37k 100999 23 sep 19:20  index-1de299e5-2cdd-40ca-8b1d-21e8aa604ff8.wt
.rw-------  37k 100999 23 sep 19:12  index-6b331759-2620-4765-b5f7-c73752fb2804.wt
.rw-------  25k 100999 23 sep 21:12  index-7a1b2b5a-a410-4f73-956a-06b970eaa3fd.wt
.rw-------  20k 100999 23 sep 19:19  index-42222a16-829e-4758-8011-89976ed4f24e.wt
.rw-------  25k 100999 23 sep 21:12  index-a9236344-77c8-4190-b276-9b23d906b665.wt
.rw-------    2 100999 23 sep 19:19  mongod.lock
.rw-------  37k 100999 23 sep 21:12  sizeStorer.wt
.rw-------  114 100999 23 sep 16:15  storage.bson
.rw-------   50 100999 23 sep 16:15  WiredTiger
.rw-------   21 100999 23 sep 16:15  WiredTiger.lock
.rw------- 1.6k 100999 23 sep 21:14  WiredTiger.turtle
.rw-------  78k 100999 23 sep 21:14  WiredTiger.wt
.rw------- 4.1k 100999 23 sep 19:19  WiredTigerHS.wt

cesar ~/Documentos/mongo-pod 21:14
→
```